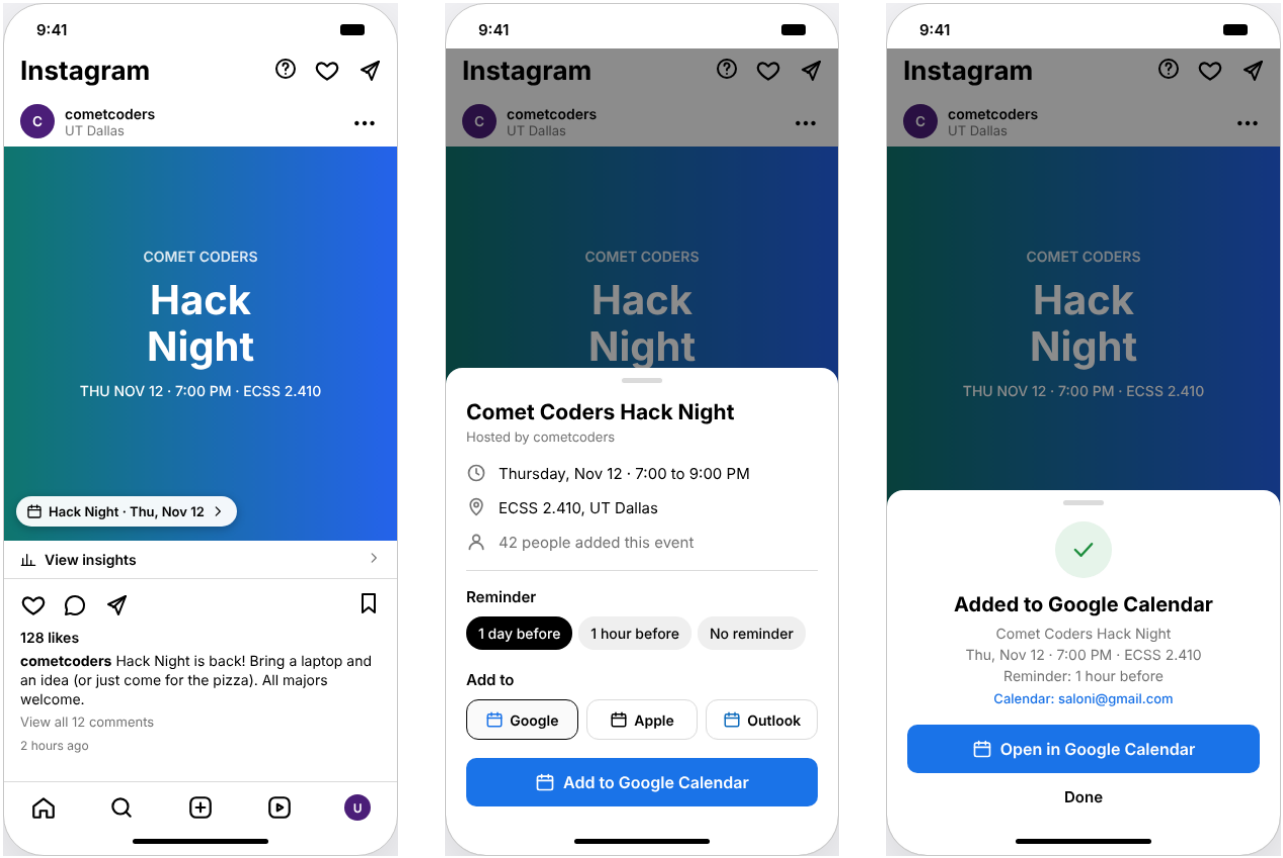
Instagram Event Tags

Build log: every step I took and every bug I fixed, from idea to a live app

Live app: insta-events-ws86.onrender.com **Code:** github.com/salonisanaroy-dotcom/insta-events

By Saloni Roy, UT Dallas, Computer Science + Economics. Fall 2026.



The one-line version

Students miss club events because a flyer post is easy to scroll past and forget. I added an **Event tag** to Instagram posts: tap it, pick a reminder, and the event goes straight into your Google, Apple, or Outlook Calendar. If the club changes the time or room, your calendar updates by itself. I designed it as a PM project, tested it with real users, then built and launched the real thing.

1. The product side

The problem

Clubs announce events on Instagram. Students see the flyer, maybe screenshot it, and forget. Nothing connects the post to the place students actually plan their week: their calendar.

Research and planning

- Interviews with students, turned into empathy maps.
- Functional and non-functional requirements, and user flows for students and for clubs.
- A paper prototype, then Figma screens.
- First plan was a calendar *inside* Instagram. I switched to Google Calendar because students already use it every day.

Testing V1 with real users

I built V1 and ran three recorded feedback sessions with no help from me. Here's what they hit and what I changed for V2:

What users said or did	What it told me	What changed in V2
"Can it add to Apple Calendar?"	Not everyone uses Google.	Added Apple and Outlook through downloadable .ics files, with the reminder built in.
Didn't know the tag was tappable	The main feature was invisible.	First-visit walkthrough, a tooltip on the first tag, and a pulse animation.
Tapped the bookmark (save) first	Saving is the habit people already have.	Saving a post with an event now asks: add it to your calendar too?
Create form looked already filled in	Example text looked like real answers.	Placeholders now start with "e.g." in gray italics, and dates and times are required.
Every club looked the same in the calendar	Hard to tell events apart at a glance.	Each club account always gets its own Google Calendar color.

Also changed along the way

- Wording: the confirmation said "Added to Google Calendar" when V1 only opened a pre-filled Google page. I changed it to "Save to Google Calendar" until the backend made "Added" true.
- The demo post was switched from a real club to a made-up one (Comet Coders Hack Night) so the project stands on its own.

2. How I built it

Stage	What I built	What it made possible
Frontend	React app (Vite) that looks like Instagram: feed, event tag, event card, creator flow (new post, add event, review, insights, edit).	A clickable prototype I could test with users.
Day 1	Node + Express server, connected to React through Vite's proxy.	The app has a backend it can talk to.
Day 2	Sign in with Google (OAuth). The server saves the login keys; the browser never sees them.	The app knows who you are and can act on your calendar.
Day 3	Google Calendar API: the server adds the event itself, with your reminder and the club's color. Adding twice updates instead of duplicating.	One tap really puts the event in your calendar.
Day 4	SQLite database (users, sessions, posts, adds) and a login cookie for each person.	Posts survive a refresh. Many people can sign in. Edits update everyone's calendar.
Extras	Delete posts (also removes the event from calendars), photo shrinking, live insights, README with screenshots.	A complete, showable product.
Launch	Hosted on Render, database moved to Turso, Google Cloud set up for the live address.	A real link anyone can open, no laptop needed.
Real users	Friends signed in, added events, and made their own posts. Two bugs found and fixed the same day.	Proof it works outside my own testing.

How a tap becomes a calendar event

- The browser sends the server only the post's id and the reminder.
- The server checks the login cookie to see who you are, then reads the event details from the database (so nobody can fake them from their browser).
- It signs in to Google as you with your saved keys and adds or updates the event.
- It saves which Google event belongs to which post, so later edits and deletes reach your calendar.

Tools

React, Vite, plain CSS, Node.js, Express, SQLite (hosted on Turso), Google OAuth 2.0, Google Calendar API, .ics calendar files, Git and GitHub, Render.

3. Every bug and how I fixed it

Grouped by where they happened. Most of these taught me something I'll use again.

Setting up

What went wrong	Why	How I fixed it
"not a git repository"	Git hadn't been started in the project folder.	Ran `git init` in the project folder, then committed.
Files unzipped into my home folder	Ran unzip from the wrong folder.	Moved the files into the project and learned to `cd` into the project first.
Website opened on port 5174, not 5173	An old terminal was still running the site.	Closed the old terminals so only one copy runs.
Terminal showed dozens of "command not found" errors	I pasted instructions along with the commands, so the terminal tried to run every sentence.	Only copy what's inside code boxes, one box at a time.

Frontend polish

What went wrong	Why	How I fixed it
Event tag text was white on white	The tag inherited the flyer's white text color.	Gave the tag its own text color.
Reminder choice reset when reopening	The card always started on the default.	The app remembers the last reminder you picked.
Tooltip floated on top of pop-ups	It showed no matter what screen was open.	Only show it on the plain feed.
"1 calendars"	The word was always plural.	Small plural helper: 1 calendar, 2 calendars.
Avatar letter showed "U"	No initial was stored for the account.	Added an initial to each post's author.

Backend

What went wrong	Why	How I fixed it
Lost my place after signing in with Google	Going to Google's page reloads the app, which wipes its memory.	Save the event you were adding before leaving, then finish it when you come back.
Only one person could be signed in	Logins were saved in one file that the next sign-in overwrote.	Database table for users plus a separate session cookie for each browser.
New posts disappeared on refresh	Posts only lived in the browser's memory.	Saved posts in the database.
Deleted demo post came back after restart	The starter post was added whenever the feed was empty.	Only add it when the database is brand new.

What went wrong	Why	How I fixed it
Feed failed with ECONNREFUSED	The server wasn't running, only the website was.	The app needs two terminals: server and website. Added a "can't reach the server" message in the feed.

Going live

What went wrong	Why	How I fixed it
Database would be erased constantly	Render's free plan wipes files on every restart and after 15 minutes of no visitors.	Moved the database to Turso (hosted SQLite). The laptop still works with a local file.
Turso dashboard said 0 rows	Its usage numbers lag behind.	Checked by querying the database directly from the terminal. The data was there.
Google sign in: redirect_uri_mismatch	The live server was sending Google an old address without the "-ws86" Render added. The setting had never saved.	Read Google's error details to see the exact address, fixed the setting on Render, redeployed.
Sign in got stuck after clicking Continue	FRONTEND_URL was missing, so the server sent people back to localhost after sign in.	Added the setting, then checked the server log said the real address before testing.
"Google hasn't verified this app"	The app is in testing mode, which is normal for a student project.	Kept testing mode, added friends as test users, and told them to click Continue.

Real users

What went wrong	Why	How I fixed it
App said "Added" but the event wasn't in my calendar, and the link said "Could not find the requested event"	Google keeps deleted events as hidden "cancelled" copies. To avoid duplicates, my app updated the old event, which stayed hidden. I confirmed it by finding two copies in Google Calendar's bin.	Before updating, the server asks Google if the event was cancelled. If so, it adds a fresh copy. The link also names the right Google account, for browsers signed into several.
A friend got "Couldn't add it automatically"	Server log: "insufficient authentication scopes." She signed in with the calendar permission box unchecked.	The server checks the permissions at sign in. If calendar access is missing, the app asks again with a note to check the box.
"invalid_grant" in the log	The sign-in page loaded twice, and Google's one-time code can only be used once.	Harmless. Nothing to fix.

What I learned

- **Read the actual error.** Google's error details and the server logs pointed straight at every deploy bug.
- **Settings are code too.** Three of the launch bugs were missing or unsaved settings, not broken code.
- **Test with other people.** Both of the last bugs only showed up when someone else used the app in a way I didn't.
- **Outside APIs have edge cases.** "Deleted" doesn't always mean gone, and users can say no to permissions.

4. What's next

- **Creator accounts.** Posts belong to whoever made them, show their name, and only they can edit or delete them. Matters now that friends are posting.
- **AI reads the flyer.** When a club uploads a flyer, AI fills in the event form for them to check. Later: a "detected event" tag on any flyer, which clubs can claim to make it verified with live updates.
- **Second round of testing** on the live app, to see if the V2 changes fixed the discoverability problem.
- **Notifications** when an event you added changes.

